

- 1. Install dependencies

```
1 import torch
2 print("torch:", torch.__version__, "cuda:", torch.version.cuda, "available:",
    torch.cuda.is_available())
```

```
torch: 2.11.0+cu128  cuda: 12.8  available: True
torch: 2.11.0+cu128  cuda: 12.8  available: True
```

```
1 TORCH = torch.__version__.split('+')[0]
2 CUDA = 'cu' + torch.version.cuda.replace('.', '') if torch.version.cuda else
  'cpu'
3
4 !pip install -q torch-geometric
5 !pip install -q pyg_lib torch_scatter torch_sparse torch_cluster
  torch_spline_conv \
6   -f https://data.pyg.org/whl/torch-{TORCH}+{CUDA}.html
7 !pip install -q optuna pyyaml tqdm scikit-learn pandas numpy scipy
8
9 import torch_geometric
10 print("torch-geometric:", torch_geometric.__version__)
```

```

_____ 64.4/64.4 kB 4.0 MB/s eta 0:00:00
_____ 64.4/64.4 kB 4.0 MB/s eta 0:00:00
_____ 1.3/1.3 MB 41.7 MB/s eta 0:00:00
_____ 1.3/1.3 MB 41.7 MB/s eta 0:00:00
_____ 5.1/5.1 MB 40.9 MB/s eta 0:00:00
_____ 5.1/5.1 MB 40.9 MB/s eta 0:00:00
_____ 10.9/10.9 MB 112.6 MB/s eta 0:00:00
_____ 10.9/10.9 MB 112.6 MB/s eta 0:00:00
_____ 5.4/5.4 MB 139.6 MB/s eta 0:00:00
_____ 5.4/5.4 MB 139.6 MB/s eta 0:00:00
_____ 3.5/3.5 MB 85.5 MB/s eta 0:00:00
_____ 3.5/3.5 MB 85.5 MB/s eta 0:00:00

```

```
Preparing metadata (setup.py) ... done
Preparing metadata (setup.py) ... done
Building wheel for torch_spline_conv (setup.py) ... done
Building wheel for torch_spline_conv (setup.py) ... done
===== 425.6/425.6 kB 15.2 MB/s eta 0:00:00
===== 265.9/265.9 kB 29.1 MB/s eta 0:00:00
===== 425.6/425.6 kB 15.2 MB/s eta 0:00:00
===== 265.9/265.9 kB 29.1 MB/s eta 0:00:00
```

```
torch-geometric: 2.8.0.post1
torch-geometric: 2.8.0.post1
```

- 2. Write the project files (embedded, no upload needed)

Every `.py` and `.yaml` file in the project is base64-encoded below and written directly to /content/grn_gnn/. This cell is idempotent - safe to re-run any time you want to reset to a known-good state, including after a runtime restart.

```

1 import base64, os
2
3 PROJECT_ROOT = '/content/grn_gnn'
4 os.makedirs(PROJECT_ROOT, exist_ok=True)
5 os.makedirs(os.path.join(PROJECT_ROOT, 'src'), exist_ok=True)
6 os.makedirs(os.path.join(PROJECT_ROOT, 'configs'), exist_ok=True)
7 os.makedirs(os.path.join(PROJECT_ROOT, 'checkpoints'), exist_ok=True)
8
9 FILES_B64 = {}
10
11 FILES_B64["src/__init__.py"] = ""
12 FILES_B64["src/data_utils.py"] = "IiIiCmRhdGFfdXRpbHMucHkKckxvYWRlcnMgZm9yIEJFRXUJTkUtZm9ybwF0IHNPbmdsZS1jZWxsIGRhdGFzZXRxZi
13 FILES_B64["src/graph_builder.py"] = "IiIiCmddyYXBoX2J1aWxkZXIucHkKckJ1aWxkcyB0aGUGY2FuZG1kYXRlIHJlZ3VsYXRvcnkgZ3JhcGgyYW5kIG
14 FILES_B64["src/model.py"] = "IiIiCm1vZGVsLnB5CgpUZw1wb3JhbCBHcmFwaCBbdHR1bnRpb24gKyBHU1Ugbw9kZWwgZm9yIEdStiBkew5hbWljcyBwcn
15 FILES_B64["src/train.py"] = "IiIiCnRyYWluLnB5CgpUcmFpbmluZyBsb29wIGZvcjBUZw1wb3JhbEdBEVdSVS4gTG9zcyA9IGV4cHJlcn3Npb24gZm9yZW
16 FILES_B64["src/evaluate.py"] = "IiIiCmV2YXx1YXRlLnB5CgpjbXBsZW11bnRzIHROZSBtZXRYaWNzIG5hbWVkIGluIHROZSBhYnN0cmFjdC9TZWN0aW9
17 FILES_B64["src/hypertune.py"] = "IiIiCmh5cGVydHVuZS5weQoKT3B0dW5hIHNL1YXJjaCBvdmdVYiHROZSB0eXBlcnBhcmFtZXRLcnMgdGhhdCBtYXR0ZX
18 FILES_B64["src/run_ablations.py"] = "IiIiCnJ1b19hYmxhdGlvbnMucHkKckRyYWluYyBhbmQgZXZhbHVhdGVzIHROZSBmb3VyIGNvbmZpZ3VyYXRpb2
19 FILES_B64["src/interpretability.py"] = "IiIiCmludGVycHJldGFiaWxpdkHkucHkKClNlY3Rpb24gNi420iBjaGVja3Mgd2hldGhlcjB0aGUGZnVsbcE
20 FILES_B64["src/score_baseline.py"] = "IiIiCnNjb3JlX2Jhc2VsaW5lLnB5CgpTY29yZXMgYSBiYXN1bGluZSBtZXRob2QncyBPVVRQVWVQgKEGecmFua
21 FILES_B64["configs/default.yaml"] = "IyBZWZhdWx0IGNvbmZpZyBmb3JgYSBzaW5nbGUGdHJhaW5pbmMcgcncVuLiBDb3B5IHBlcn1kYXRkZm90IChorV
22

```

```

23 for rel_path, b64_content in FILES_B64.items():
24     full_path = os.path.join(PROJECT_ROOT, rel_path)
25     decoded = base64.b64decode(b64_content)
26     with open(full_path, 'wb') as f:
27         f.write(decoded)
28
29 print(f"wrote {len(FILES_B64)} files to {PROJECT_ROOT}")
30 for rel_path in FILES_B64:
31     full_path = os.path.join(PROJECT_ROOT, rel_path)
32     print(f" {rel_path}: {os.path.getsize(full_path)} bytes")

```

```

wrote 11 files to /content/grn_gnn
src/__init__.py: 0 bytes
src/data_utils.py: 6477 bytes
src/graph_builder.py: 3487 bytes
src/model.py: 6192 bytes
src/train.py: 6772 bytes
src/evaluate.py: 4209 bytes
src/hypertune.py: 4536 bytes
src/run_ablations.py: 4114 bytes
src/interpretability.py: 5665 bytes
src/score_baseline.py: 3929 bytes
configs/default.yaml: 733 bytes

```

```

1 import sys, os
2
3 # always work from the correct root, always re-verify import state fresh
4 os.chdir("/content/grn_gnn")
5 if "/content/grn_gnn" not in sys.path:
6     sys.path.insert(0, "/content/grn_gnn")
7
8 for mod_name in list(sys.modules):
9     if mod_name == "src" or mod_name.startswith("src."):
10         del sys.modules[mod_name]
11
12 # byte-level sanity check -- confirms the gene-cap fix is actually present
13 # before we do anything else, so we never silently repeat last session's bug
14 with open("/content/grn_gnn/src/data_utils.py", "rb") as f:
15     _content = f.read()
16 assert b"priority = [g for g in ranked" in _content, \
17     "gene-cap fix missing from written file -- setup cell did not run correctly"
18 print("verified: gene-cap fix present in data_utils.py")
19 print("cwd:", os.getcwd())

```

```

verified: gene-cap fix present in data_utils.py
cwd: /content/grn_gnn
verified: gene-cap fix present in data_utils.py
cwd: /content/grn_gnn

```

✓ 3. Get BEELINE data (expression + pseudotime + ground-truth networks + TF list)

Downloads from Zenodo (the GitHub repo only has preprocessing scripts, not the processed data -- see earlier troubleshooting).

```

1 !wget -q https://zenodo.org/records/3701939/files/BEELINE-data.zip -O BEELINE-data.zip
2 !wget -q https://zenodo.org/records/3701939/files/BEELINE-Networks.zip -O BEELINE-Networks.zip
3 !unzip -oq BEELINE-data.zip -d beeline_data
4 !unzip -oq BEELINE-Networks.zip -d beeline_networks
5 print("done")

```

```

done
done

```

```

1 import os, shutil, pandas as pd
2
3 DATASET = "hESC"
4 expr_src_dir = "beeline_data/BEELINE-data/inputs/scRNA-Seq/hESC"
5 net_src_file = "beeline_networks/Networks/human/hESC-ChIP-seq-network.csv"
6 tf_list_file = "beeline_networks/human-tfs.csv"
7
8 dst_dir = f"data/BEELINE/{DATASET}"
9 os.makedirs(dst_dir, exist_ok=True)
10
11 shutil.copy(f"{expr_src_dir}/ExpressionData.csv", f"{dst_dir}/ExpressionData.csv")
12 shutil.copy(f"{expr_src_dir}/PseudoTime.csv", f"{dst_dir}/PseudoTime.csv")
13

```

```

14 ref = pd.read_csv(net_src_file)
15 ref.to_csv(f"{dst_dir}/refNetwork.csv", index=False)
16
17 tf_df = pd.read_csv(tf_list_file)
18 tf_list = tf_df["TF"].tolist()
19
20 print(os.listdir(dst_dir))
21 print(ref.shape, "ground-truth edges")
22 print(len(tf_list), "TFs")

['PseudoTime.csv', 'ExpressionData.csv', 'refNetwork.csv']
(436563, 2) ground-truth edges
1563 TFs

```

4. Priors (TF binding + chromatin accessibility) -- optional, needed for the real full-model run

Upload `tf_binding.csv` (columns: `TF,Target,Score`) and `chromatin_access.csv` (columns: `Gene,Score`) once sourced. Until then, leave `use_priors=False` and you're running the no-priors ablation.

```

1 os.makedirs("data/priors", exist_ok=True)
2 # from google.colab import files
3 # uploaded_priors = files.upload()
4 # for fname in uploaded_priors:
5 #     shutil.move(fname, f"data/priors/{fname}")
6
7 HAVE_PRIORS = False # flip to True once the two files above are in data/priors/

```

5. Build config

```

1 import yaml
2
3 with open("configs/default.yaml") as f:
4     cfg = yaml.safe_load(f)
5
6 cfg["dataset_name"] = DATASET
7 cfg["data_dir"] = dst_dir
8 cfg["tf_list"] = tf_list
9 cfg["tf_binding_path"] = "data/priors/tf_binding.csv" if HAVE_PRIORS else None
10 cfg["chromatin_path"] = "data/priors/chromatin_access.csv" if HAVE_PRIORS else None
11 cfg["use_priors"] = HAVE_PRIORS
12 cfg["top_n_genes"] = 1000
13 cfg["device"] = "cuda" if torch.cuda.is_available() else "cpu"
14 cfg["checkpoint_path"] = f"checkpoints/model_{DATASET}.pt"
15 cfg["batch_size"] = 2 # keep small -- see memory notes from earlier sessions
16 cfg["hidden_dim"] = 64
17 cfg["heads"] = 4
18 cfg["gat_layers"] = 1
19 cfg["window"] = 7
20 cfg["lr"] = 0.0007704764581074137
21 cfg["dropout"] = 0.28687457289899715
22 cfg["weight_decay"] = 8.445718293202698e-05
23 cfg["prior_reg_weight"] = 0.08436661565904713
24 cfg["max_epochs"] = 200
25 cfg["patience"] = 15
26
27 with open("configs/current.yaml", "w") as f:
28     yaml.safe_dump(cfg, f)
29
30 print(yaml.safe_dump(cfg))

```

```

batch_size: 2
checkpoint_path: checkpoints/model_hESC.pt
chromatin_path: null
corr_topk: 20
data_dir: data/BEELINE/hESC
dataset_name: hESC
device: cuda
dropout: 0.28687457289899715
gat_layers: 1
heads: 4
hidden_dim: 64
lr: 0.0007704764581074137
max_epochs: 200

```

```

patience: 15
prior_reg_weight: 0.08436661565904713
tf_binding_path: null
tf_list:
- AATF
- ABL1
- ABT1
- ABTB1
- ADNP
- AEBP1
- AEBP2
- AES
- AFF1
- AFF4
- AHR
- AIP
- AIRE
- AKAP17A
- AKT1
- AKT2
- AKT3
- ALS2CR8
- ALX1
- ALX4
- ALYREF
- ANKRA2
- ANKRD1
- ANKRD2
- ANKRD6
- ANKRD7
- ANKS1A
- ANKS3
- ANKZF1
- APBB1
- APBB2
- APC
- APEX1
- AR
- ARHGAP35
- ARID1A
- ARID1B
- ARID2
- ARID3A
- ARID3B
- ARID4A

```

✓ Sanity check: candidate-edge recall before training

Confirms the TF-based candidate construction and the gene cap are both working before you spend GPU time training on them.

```

1 from src.data_utils import load_beeline_dataset
2 from src.graph_builder import build_candidate_edges
3 from src.evaluate import edges_to_binary_labels
4
5 ds_check = load_beeline_dataset(data_dir=cfg["data_dir"], name=cfg["dataset_name"],
6                                 tf_binding_path=None, chromatin_path=None,
7                                 top_n_genes=cfg["top_n_genes"])
8 print("n_genes:", len(ds_check.genes))
9
10 edges_check = build_candidate_edges(ds_check.genes, tf_list=cfg["tf_list"], expr=ds_check.expr)
11 labels_check = edges_to_binary_labels(edges_check, ds_check.genes, ds_check.ref_edges)
12
13 gene_set = set(ds_check.genes)
14 reachable = ds_check.ref_edges[ds_check.ref_edges["Gene1"].isin(gene_set) &
15                               ds_check.ref_edges["Gene2"].isin(gene_set)]
16
17 print("candidate edges:", len(labels_check))
18 print("true edges recovered:", labels_check.sum())
19 print("true edges reachable (both endpoints in gene set):", len(reachable))
20 print("recall against reachable ceiling:", labels_check.sum() / len(reachable) if len(reachable) else "n/a")
21
22 assert len(ds_check.genes) == cfg["top_n_genes"], \
23        f"expected {cfg['top_n_genes']} genes, got {len(ds_check.genes)} -- gene cap not applied correctly"
24 print("\nOK to proceed to training.")

```

```

n_genes: 1000
n_genes: 1000
candidate edges: 97902
true edges recovered: 2157
true edges reachable (both endpoints in gene set): 2239

```

```
recall against reachable ceiling: 0.9633765073693613
```

```
OK to proceed to training.  
candidate edges: 97902  
true edges recovered: 2157  
true edges reachable (both endpoints in gene set): 2239  
recall against reachable ceiling: 0.9633765073693613
```

```
OK to proceed to training.
```

6. Train the main config

This retrains on the corrected data. Expect early stopping somewhere before `max_epochs`; watch for the "early stopping at epoch N" line.

```
1 from src.train import run_training  
2  
3 model, ds, edge_index, edge_attr, (X_te, Y_te), best_val = run_training(cfg)  
4 torch.save(model.state_dict(), cfg["checkpoint_path"])  
5 print("final val loss:", best_val)
```

```
epoch 000 train_loss 10.3352 val_loss 11.5898  
epoch 005 train_loss 8.1461 val_loss 10.0345  
epoch 010 train_loss 7.9874 val_loss 9.7951  
epoch 015 train_loss 7.9425 val_loss 9.8381  
epoch 020 train_loss 7.9269 val_loss 9.8087  
epoch 025 train_loss 7.9212 val_loss 9.7844  
epoch 030 train_loss 7.9177 val_loss 9.7163  
epoch 035 train_loss 7.9050 val_loss 9.7207  
epoch 040 train_loss 7.8760 val_loss 9.7718  
epoch 045 train_loss 7.8641 val_loss 9.7124  
epoch 050 train_loss 7.8326 val_loss 9.7159  
epoch 055 train_loss 7.8423 val_loss 9.7034  
epoch 060 train_loss 7.8453 val_loss 9.7495  
epoch 065 train_loss 7.8365 val_loss 9.7111  
epoch 070 train_loss 7.8441 val_loss 9.7306  
epoch 075 train_loss 7.8172 val_loss 9.6890  
early stopping at epoch 77  
final val loss: 9.672511624438423
```

7. Evaluate: topology recovery + robustness

```
1 from src.evaluate import evaluate_topology_recovery, robustness_sweep  
2 from src.graph_builder import build_candidate_edges  
3  
4 edges = build_candidate_edges(ds.genes, tf_list=cfg["tf_list"], expr=ds.expr)  
5 x_window = torch.tensor(X_te[0], dtype=torch.float32, device=edge_index.device)  
6  
7 topo_metrics = evaluate_topology_recovery(model, ds, edge_index, edge_attr, edges, x_window)  
8 print("Topology recovery:", topo_metrics)  
9  
10 robustness_df = robustness_sweep(model, edge_index, edge_attr, X_te, Y_te,  
11                                 device=edge_index.device)  
12 robustness_df.to_csv("robustness_results.csv", index=False)  
13 print(robustness_df)
```

```
Topology recovery: {'auprc': np.float64(0.025308442894800956), 'auroc': np.float64(0.5354079116959787), 'epr': np.float64(1.4939  
perturbation severity mse  
0 dropout 0.00 7.677816  
1 dropout 0.10 8.268546  
2 dropout 0.20 9.168583  
3 dropout 0.30 10.503317  
4 dropout 0.40 12.088186  
5 gaussian_noise 0.00 7.677816  
6 gaussian_noise 0.05 7.677689  
7 gaussian_noise 0.10 7.681858  
8 gaussian_noise 0.20 7.691313
```

8. Run all four ablations (Section 6.5 / Table 2)

Trains `full_model`, `no_priors`, `no_temporal`, and `neither` from scratch, each saved as a separate checkpoint. Takes roughly 4x as long as a single training run -- expect this to take a while.

```

1 from src.run_ablations import run_all_ablations
2
3 ablation_df = run_all_ablations(cfg)
4 ablation_df.to_csv("ablation_results.csv", index=False)
5 ablation_df

```

```

=== running ablation: full_model (use_priors=True, use_temporal=True) ===
WARNING: use_priors=True but no tf_binding_path set in config; edge features will be all-zero, which is NOT a real 'full model'
epoch 000  train_loss 9.8583  val_loss 10.2697
epoch 005  train_loss 8.4208  val_loss 9.9626
epoch 010  train_loss 8.4815  val_loss 9.8224
epoch 015  train_loss 8.7741  val_loss 9.7812
epoch 020  train_loss 8.8954  val_loss 9.7357
epoch 025  train_loss 8.9019  val_loss 9.7581
epoch 030  train_loss 8.8548  val_loss 9.7057
epoch 035  train_loss 8.7689  val_loss 9.7102
early stopping at epoch 37
saved checkpoint: checkpoints/model_hESC_full_model.pt
full_model: AUPRC=0.0267 AUROC=0.5598 EPR=1.3677 MSE=7.7263

=== running ablation: no_priors (use_priors=False, use_temporal=True) ===
epoch 000  train_loss 9.9575  val_loss 10.4057
epoch 005  train_loss 7.9825  val_loss 9.7878
epoch 010  train_loss 7.9380  val_loss 9.9530
epoch 015  train_loss 7.9087  val_loss 9.8392
epoch 020  train_loss 7.8820  val_loss 9.7977
epoch 025  train_loss 7.8809  val_loss 9.7413
epoch 030  train_loss 7.8714  val_loss 9.7463
epoch 035  train_loss 7.8455  val_loss 9.8652
epoch 040  train_loss 7.8648  val_loss 9.7107
epoch 045  train_loss 7.8431  val_loss 9.7326
epoch 050  train_loss 7.8386  val_loss 9.6905
epoch 055  train_loss 7.8477  val_loss 9.6878
epoch 060  train_loss 7.8409  val_loss 9.6811
epoch 065  train_loss 7.8586  val_loss 9.7155
epoch 070  train_loss 7.8263  val_loss 9.6970
epoch 075  train_loss 7.8219  val_loss 9.6812
early stopping at epoch 79
saved checkpoint: checkpoints/model_hESC_no_priors.pt
no_priors: AUPRC=0.0229 AUROC=0.4669 EPR=1.5150 MSE=7.7012

=== running ablation: no_temporal (use_priors=True, use_temporal=False) ===
WARNING: use_priors=True but no tf_binding_path set in config; edge features will be all-zero, which is NOT a real 'full model'
epoch 000  train_loss 10.1377  val_loss 11.3342
epoch 005  train_loss 9.8822  val_loss 11.2501
epoch 010  train_loss 9.8961  val_loss 11.0534
epoch 015  train_loss 9.8219  val_loss 11.1629
epoch 020  train_loss 9.7472  val_loss 11.0583
epoch 025  train_loss 9.7204  val_loss 11.0683
early stopping at epoch 26
saved checkpoint: checkpoints/model_hESC_no_temporal.pt
no_temporal: AUPRC=0.0243 AUROC=0.4937 EPR=1.7255 MSE=9.3424

=== running ablation: neither (use_priors=False, use_temporal=False) ===
epoch 000  train_loss 9.7978  val_loss 11.2468
epoch 005  train_loss 9.2637  val_loss 11.0636
epoch 010  train_loss 9.2342  val_loss 11.0161
epoch 015  train_loss 9.2060  val_loss 11.0636
epoch 020  train_loss 9.2299  val_loss 11.1081
epoch 025  train_loss 9.1877  val_loss 11.0300
epoch 030  train_loss 9.2017  val_loss 11.0822
early stopping at epoch 31
saved checkpoint: checkpoints/model_hESC_neither.pt
neither: AUPRC=0.0314 AUROC=0.5909 EPR=2.0832 MSE=9.2149

```

9. Interpretability analysis (Section 6.6)

	combined	use_priors	use_temporal	auprc	auroc	epr	forecasting_mse	val_loss
0	full_model	True	True	0.026733	0.559802	1.367743	7.726305	9.693864
1	no_priors	False	True	0.022949	0.466907	1.515039	7.701170	9.668475

Requires the full_model and no_priors checkpoints from step 8.

```

1 from src.interpretability import run_interpretability_analysis
2
3 full_ckpt = cfg["checkpoint_path"].replace(".pt", "_full_model.pt")
4 no_priors_ckpt = cfg["checkpoint_path"].replace(".pt", "_no_priors.pt")
5
6 combined, edge_scores = run_interpretability_analysis(cfg, full_ckpt, no_priors_ckpt)

```

k	true_edges_in_top_k	precision_at_k	model
50	2	0.040	full_model
100	3	0.030	full_model
250	8	0.032	full_model
500	11	0.022	full_model

50	2	0.040	no_priors
100	3	0.030	no_priors
250	8	0.032	no_priors
500	11	0.022	no_priors

saved interpretability_topk_precision.csv and interpretability_edge_scores.csv

10. Baseline scoring (Section 6.2 / Table 1)

SCORE, GRISLI, SCNS, GENELink, and GNNLink are other groups' published methods -- run each one's own official code separately (BEELINE ships Docker images for SCORE/GRISLI/SCNS; GENELink/GNNLink have their own GitHub repos), export each one's predicted edges to a CSV with columns `(Gene1, Gene2, Score)`, then score it here with the same metrics used above.

```
1 from src.score_baseline import score_baseline_output
2
3 # example -- repeat once per baseline, once you have its output CSV:
4 baseline_metrics = score_baseline_output(
5     scores_csv="results/gnnlink_hesc_scores.csv",
6     ds=ds, baseline_name="GNNLink",
7 )
8 print(baseline_metrics)
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_1929/1237607354.py in <cell line: 0>()
      2
      3 # example -- repeat once per baseline, once you have its output CSV:
----> 4 baseline_metrics = score_baseline_output(
      5     scores_csv="results/gnnlink_hesc_scores.csv",
      6     ds=ds, baseline_name="GNNLink",
      7 )

----- 5 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map,
is_text, errors, storage_options)
    871     if ioargs.encoding and "b" not in ioargs.mode:
    872         # Encoding
--> 873         handle = open(
    874             handle,
    875             ioargs.mode,

FileNotFoundError: [Errno 2] No such file or directory: 'results/gnnlink_hesc_scores.csv'
```

11. Save everything to Drive before the session ends

Colab's local disk doesn't persist across sessions.

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3 import shutil
4 shutil.copytree("checkpoints", "/content/drive/MyDrive/grn_gnn_checkpoints", dirs_exist_ok=True)
5 for f in ["ablation_results.csv", "robustness_results.csv",
6         "interpretability_topk_precision.csv", "interpretability_edge_scores.csv",
7         "configs/current.yaml"]:
8     if os.path.exists(f):
9         shutil.copy(f, "/content/drive/MyDrive/")
10 print("saved to Drive")
```

Mounted at /content/drive
saved to Drive

